

Как да напишем `deerscore.cpp`

ръководство стъпка по стъпка

Microsoft Visual Studio · C++17 · деструктор, дълбоко копиране, оператор =

Бележка за условието. Сред файловете на курса няма отделно условие за упражнение 3 — файлът `tema_03.pdf` е точно копие на `tema_02.pdf` (еднакви до байт). Условието на упражнение 4 обаче се позовава на „класа `CAddress` от упражнение 3“, „класа `CPerson` от упражнение 3“ и „класа `CStudent` от упражнение 3“ — тоест упражнение 3 съществува и надгражда същите три класа.

Това ръководство **възстановява** упражнение 3 по темата, която стои между наследяването (упр. 2) и приятелските оператори (упр. 4) и без която упражнение 4 не може да се напише коректно: динамична памет в класа. Ако получите оригиналното условие, сравнете с него — **класовете и кодът остават съвместими с упражнение 4 във всички случаи**, защото тук нищо не се маха, а само се добавя.

Какво ще направим

До упражнение 2 всички член-променливи бяха `string` и `int` — типове, които се грижат сами за паметта си. Затова генерираният от компилатора копиращ конструктор работеше правилно и ние го написахме само за да го видим.

Сега към `CStudent` добавяме **динамичен масив от оценки**: `double* m_pdGrades;`. В мига, в който класът започне да държи *указател* към своя памет, генерираните от компилатора функции стават грешни — и програмата се чупи по начини, които не личат от кода.

Какво добавяме	Какъв проблем решава
Деструктор <code>~CStudent()</code>	Заделената с <code>new[]</code> памет не се освобождава сама. Без деструктор — теч на памет.
Копиращ конструктор (дълбок)	Генерираният копира <i>указателя</i> , а не масива. Двата обекта започват да сочат към една и съща памет.
Оператор <code>operator=</code>	Същият проблем при <code>a = b;</code> , плюс изтичане на старата памет на <code>a</code> .

Тези три функции вървят заедно. Правилото е известно като **правилото на тримата**: ако ви е потребвала една от тях, почти сигурно ви трябва и трите.

План на работата

1. Нов проект и класовете от упражнение 2
2. Проблемът, видян на живо — плитко копиране
3. Динамичният масив в CStudent
4. Деструктор (първият от тримата)
5. Копиращ конструктор с дълбоко копиране (вторият)
6. Оператор за присвояване (третият)
7. Работа с оценките — AddGrade и растеж на масива
8. Извеждане и главна функция

1 Нов проект и класовете от упражнение 2

Направете нов **Empty Project**, например `Тема3_DeepCopy`, и файл `deepcopy.cpp`.

Пренесете от упражнение 2 без нито една промяна: `enum Gender`, целия клас `CPerson` и целия клас `CAddress`. Класът `CStudent` също се пренася — него ще надграждаме от стъпка 3 нататък.

Проверка: компилирайте и стартирайте програмата от упражнение 2 в новия проект, преди да сте променили каквото и да било. Трябва да работи както преди.

2 Проблемът, видян на живо

Преди да пишем решението, нека видим счупеното. Ще направим нарочно непълен клас — само указател, масив и деструктор, **без** копиращ конструктор и **без** оператор `=`.

ДОБАВЕТЕ НАД ФУНКЦИЯТА MAIN

```
class CBadGrades
{
public:
    double* m_pdGrades;    // указател към динамичен масив
    size_t  m_nCount;

    CBadGrades(size_t nCount)
        : m_pdGrades(new double[nCount]), m_nCount(nCount)
    {
        for (size_t i = 0; i < nCount; i++)
            m_pdGrades[i] = 0.0;
    }

    ~CBadGrades() { delete[] m_pdGrades; }

    // НЯМА копиращ конструктор и НЯМА оператор =.
    // Компиляторът генерира свои – те копират САМИЯ УКАЗАТЕЛ.
};
```

ДОБАВЕТЕ В MAIN

```
{
    CBadGrades a(3);
    a.m_pdGrades[0] = 5.50;

    CBadGrades b(a);          // генерираният копиращ конструктор

    cout << "  a[0] преди промяната на b: " << a.m_pdGrades[0] << endl;
    b.m_pdGrades[0] = 2.00;    // пипаме CAMO b...
    cout << "  a[0] след промяната на b: " << a.m_pdGrades[0]
        << "    <-- промени се и a!" << endl;

    // Обезвреждаме двойното изтриване, за да не се срие програмата:
    b.m_pdGrades = nullptr;
}
```

КАКВО СЕ СЛУЧВА

Генерираният копиращ конструктор копира всеки член поотделно. За `m_pdGrades` това значи: копира се **числото, което е в указателя** — адресът. Резултатът:

```
a.m_pdGrades —┐
                └─→ [ 5.50 | 0.00 | 0.00 ]    ЕДИН масив
b.m_pdGrades —┐
```

Двата обекта сочат към един и същ масив. Затова промяна през `b` се вижда през `a`. Нарича се **плитко копиране** (shallow copy).

Второто, по-лошо последствие. В края на блока се викат *двата* деструктора. Всеки изпълнява `delete[] m_pdGrades;` върху един и същи адрес. Освобождаване на вече освободена памет е *неопределено поведение*: под Windows програмата се срива с `HEAP CORRUPTION DETECTED`, а в Debug режим Visual Studio показва прозорец с `_CrtIsValidHeapPointer`.

Затова в кода по-горе пишем `b.m_pdGrades = nullptr;` преди края на блока — обезвреждаме втория деструктор, за да може демонстрацията да продължи. `delete[]` върху `nullptr` е безопасно и не прави нищо.

Задача за проверка: закоментирайте реда `b.m_pdGrades = nullptr;` и стартирайте с `F5`. Програмата ще се срие в края на блока. Разгледайте съобщението, после върнете реда. Това е грешката, срещу която пишем останалите стъпки.

3 Динамичният масив в CStudent

ДОБАВЕТЕ В ЧАСТНАТА СЕКЦИЯ НА CStudent

```
private:
    string  m_strFN;
    CAddress m_address;

    // Динамичната част – това налага правилото на тримата
```

```
double* m_pdGrades;    // указател към масива с оценки
size_t  m_nCount;      // колко оценки има в момента
size_t  m_nCapacity;   // за колко има заделено място
```

ЗАЩО СА ТРИ ПРОМЕНЛИВИ, А НЕ ЕДНА

<code>m_pdGrades</code>	Указател към първия елемент. Самият масив живее в динамичната памет (heap), не в обекта. Обектът пази само адреса — 8 байта.
<code>m_nCount</code>	Колко оценки реално са записани. Масивът не знае колко от него е използван — това трябва да се пази отделно.
<code>m_nCapacity</code>	За колко елемента е заделено място. Разделено е от <code>m_nCount</code> , за да не заделяме нова памет при всяка добавена оценка.

Защо `size_t`, а не `int`? `size_t` е типът, с който езикът мери размери и брой елементи — винаги без знак и достатъчно голям за всеки възможен масив. Ако използвате `int`, при сравнение с `sizeof(...)` или с `.size()` Visual Studio дава предупреждение C4267 за загуба на данни.

ВСИЧКИ КОНСТРУКТОРИ ТРЯБВА ДА ЗАНУЛЯТ УКАЗАТЕЛЯ

```
CStudent::CStudent()
    : CPerson(), m_strFN("(няма)", m_address(),
      m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0)
{
}
```

Същите три реда се добавят и в двата експлицитни конструктора.

Това е задължително. Ако не инициализирате `m_pdGrades`, в него стои каквото е било в паметта. Деструкторът ще извика `delete[]` върху този случаен адрес и програмата ще се срине. Правилото: **всеки указател-член се инициализира с `nullptr` във всеки конструктор.**

4 Деструктор — първият от тримата

ОБЯВЕТЕ В КЛАСА

```
~CStudent();
```

НАПИШЕТЕ СЛЕД КЛАСА

```
CStudent::~~CStudent()
{
    delete[] m_pdGrades;    // delete[], защото е заделено с new[]
    m_pdGrades = nullptr;
}
```

За
деструктора

Име	Името на класа с ~ отпред.
Параметри	Няма и не може да има. Затова деструкторът е само един.
Тип на връщане	Няма — дори не <code>void</code> .
Кога се вика	Автоматично: при излизане от блока за локални обекти, при <code>delete</code> за динамични, в края на <code>main</code> за глобални.

`delete[]` и `delete` не са взаимозаменяеми. Заделено с `new double[n]` се освобождава с `delete[]`. Заделено с `new double` — с `delete`. Разминаването е неопределено поведение: компилаторът не се оплаква, а програмата се чупи по-късно и на друго място.

Защо `m_pdGrades = nullptr;` след `delete[]`? В деструктора обектът и без това умира, така че този ред не е строго нужен. Пише се като навик: същият ред в `operator=` (стъпка 6) е *задължителен*, а еднообразието пази от пропуски. Освен това при отстраняване на грешки нулевият указател се разпознава веднага, докато „висящият“ указател към освободена памет изглежда валиден.

5 Копиращ конструктор — вторият от тримата

Задачата: новият обект да получи **собствен** масив със същите стойности.

НАПИШЕТЕ СЛЕД КЛАСА

```
CStudent::CStudent(const CStudent& stud)
: CPerson(stud), m_strFN(stud.m_strFN), m_address(stud.m_address),
  m_pdGrades(nullptr), m_nCount(stud.m_nCount), m_nCapacity(stud.m_nCapacity)
{
    if (m_nCapacity > 0)
    {
        m_pdGrades = new double[m_nCapacity]; // СОБСТВЕНА памет
        for (size_t i = 0; i < m_nCount; i++) // копираме СТОЙНОСТИТЕ
            m_pdGrades[i] = stud.m_pdGrades[i];
    }
}
```

ПЛИТКО СРЕЩУ ДЪЛБОКО КОПИРАНЕ

	Плитко (генерираното)	Дълбоко (нашето)
Какво се копира	адресът	стойностите
Брой масиви след копирането	един	два
Промяна в копие	вижда се в оригинала	не се вижда
В края на живота	двойно <code>delete[]</code> → срыв	всеки трие своето

След дълбокото копиране картината е:

```
stud1.m_pdGrades → [ 5.50 | 4.00 | 6.00 ]
stud2.m_pdGrades → [ 5.50 | 4.00 | 6.00 ]    отделен масив
```

ТРИ ПОДРОБНОСТИ, КОИТО ЛЕСНО СЕ ПРОПУСКАТ

- 1 `m_pdGrades(nullptr)` стои в списъка за инициализация, макар веднага след това да му присвояваме. Ако `m_nCapacity` е 0, тялото не се изпълнява и указателят трябва вече да е нулев.
- 2 Заделяме `m_nCapacity` елемента, но копираме само `m_nCount`. Останалите са неизползвани — няма смисъл да се копират.
- 3 `CPerson(stud)` и `m_address(stud.m_address)` вършат своята работа сами. `string` има собствен коректен копиращ конструктор — за него не се грижим.

6 Оператор за присвояване — третият от тримата

Разликата спрямо копиращия конструктор: тук обектът **вече съществува** и вече държи памет. Значи има една стъпка повече — старата памет трябва да се освободи.

ОБЯВЕТЕ В КЛАСА

```
CStudent& operator=(const CStudent& stud);
```

НАПИШЕТЕ СЛЕД КЛАСА

```
CStudent& CStudent::operator=(const CStudent& stud)
{
    // 1. Защита от самоприсвояване: stud1 = stud1;
    if (this == &stud)
        return *this;

    // 2. Базовата и вградената част – чрез техните оператори =
    CPerson::operator=(stud);
    m_strFN   = stud.m_strFN;
    m_address = stud.m_address;

    // 3. Старата памет се освобождава ПРЕДИ да се заеме нова
    delete[] m_pdGrades;
    m_pdGrades = nullptr;

    // 4. Дълбоко копиране, както в копиращия конструктор
    m_nCount   = stud.m_nCount;
    m_nCapacity = stud.m_nCapacity;
    if (m_nCapacity > 0)
    {
        m_pdGrades = new double[m_nCapacity];
        for (size_t i = 0; i < m_nCount; i++)
            m_pdGrades[i] = stud.m_pdGrades[i];
    }

    // 5. Връщаме *this, за да работи веригата a = b = c;
    return *this;
}
```

ПЕТТЕ ЧАСТИ, ЕДНА ПО ЕДНА

- 1 **Проверката за самоприсвояване не е излишна педантичност.** Проследете без нея какво прави `stud1 = stud1;` : стъпка 3 освобождава масива на `stud1`; стъпка 4 после копира от `stud.m_pdGrades` — тоест от **вече изтритата** памет. Обектът се самоунищожава. `this` е адресът на обекта, върху който работим, `&stud` — на подадения; ако съвпадат, няма какво да се прави.
- 2 Базовият клас има свой `operator=` — генериран, но напълно коректен, защото `CPerson` няма указатели. Извиква се изрично с `CPerson::`, иначе наследената част остава непроменена.
- 3 Ако пропуснете `delete[]`, старият масив остава заделен, но вече недостъпен — **теч на памет**. Програмата работи и нищо не личи, докато не се повтори хиляди пъти.
- 4 Същият код като в копиращия конструктор. Дублирането е съзнателно — така двете функции се четат независимо.
- 5 `*this` е самият обект. Връщането по референция позволява `studA = studB = stud1;` — присвояването се изпълнява отдясно наляво и резултатът от вътрешното служи за дясна страна на външното.

Защо типът на връщане е `CStudent&`, а не `void`? С `void` класът пак ще работи за просто `a = b;`, но `a = b = c;` няма да се компилира. Връщането по **референция**, а не по стойност, спестява едно излишно копиране на целия обект при всяко присвояване.

Проверка: компилирайте. Ако получите `C2582: 'operator =' function is unavailable`, значи някъде има `const` член-променлива — константните членове не могат да се присвояват.

7 Работа с оценките и растеж на масива

Масивът е с фиксиран размер, а оценките се добавят една по една. Решението: когато се напълни, заделяме двойно по-голям и пренасяме старите стойности.

ОБЯВЕТЕ В КЛАСА

```
private:
    void grow();                // помощна, затова е частна

public:
    void AddGrade(double dGrade);
    double GetGrade(size_t nIndex) const;
    size_t GetGradeCount() const;
    double GetAverage() const;
```

НАПИШЕТЕ СЛЕД КЛАСА

```
void CStudent::grow()
{
    size_t nNew = (m_nCapacity == 0) ? 4 : m_nCapacity * 2;
    double* pdNew = new double[nNew];

    for (size_t i = 0; i < m_nCount; i++)    // пренасяме старите стойности
        pdNew[i] = m_pdGrades[i];
```

```

        delete[] m_pdGrades;                // освобождаваме стария масив
        m_pdGrades = pdNew;
        m_nCapacity = nNew;
    }

void CStudent::AddGrade(double dGrade)
{
    if (dGrade < 2.0 || dGrade > 6.0)        // проверка за валидност
    {
        cout << " (отхвърлена оценка: " << dGrade << ")" << endl;
        return;
    }
    if (m_nCount == m_nCapacity)
        grow();
    m_pdGrades[m_nCount] = dGrade;
    m_nCount++;
}

```

Редът в `grow` е задължителен. Първо се заделя новият масив, после се копира, и чак накрая се трие старият. Ако изтриете стария преди копирането, ще четете от освободена памет.

Защо удвояване, а не „+1“? При заделяне на един елемент повече всяко добавяне значи ново заделяне и пренасяне на всичко — за n оценки това е около $n^2/2$ копирания. При удвояване заделянията са около $\log_2 n$ на брой, а общият брой копирания остава пропорционален на n . Точно това прави и `std::vector`, който ще използваме от упражнение 9 нататък.

Сега вие: напишете `GetGradeCount` (един ред), `GetGrade` (връща `0.0` при `nIndex >= m_nCount`) и `GetAverage` (връща `0.0` при празен масив — иначе ще делите на нула). И трите са `const`. Сравнете с листинга.

8 Извеждане и главна функция

ДОПЪЛНЕТЕ CStudent::Output

```

ostream& CStudent::Output(ostream& toStream) const
{
    CPerson::Output(toStream);
    toStream << ", ф.н.: " << m_strFN << ", адрес: ";
    m_address.Output(toStream);

    toStream << ", оценки: ";
    if (m_nCount == 0)
        toStream << "(няма)";
    else
    {
        for (size_t i = 0; i < m_nCount; i++)
            toStream << m_pdGrades[i] << (i + 1 < m_nCount ? " " : "");
        toStream << " | среден успех: " << GetAverage();
    }
}

```

```
    return toStream;
}
```

8.1 Форматиране на числата

```
cout.setf(ios::fixed);
cout.precision(2);
```

Без тези два реда `5.166666...` се извежда като `5.16667`. С тях — като `5.17`, което е обичайното за оценки. Задава се веднъж в началото на `main` и важи до края.

8.2 Тест на копиращия конструктор

```
CStudent stud1("Иван Петров", addrVarna, "111111110", "21621301");
stud1.AddGrade(5.50);
stud1.AddGrade(4.00);
stud1.AddGrade(6.00);

CStudent stud2(stud1);                // копиращ конструктор
stud2.SetName("Елена Димитрова");
stud2.AddGrade(3.00);                // добавяме СМО в копието
cout << "  stud1 (оригинал): "; stud1.Output(cout) << endl;
cout << "  stud2 (копие):    "; stud2.Output(cout) << endl;
```

Ако копирането е дълбоко, `stud1` остава с три оценки, а `stud2` има четири. При плитко копиране и двата биха показали четири.

8.3 Тест на оператора за присвояване

```
CStudent stud3("Георги Стоянов", addrVarna, "1111111104", "21621303");
stud3.AddGrade(2.00);
stud3 = stud1;                        // оператор =
stud3.AddGrade(4.50);                // промяна само в stud3
cout << "  stud1 след промяна на stud3: "; stud1.Output(cout) << endl;
cout << "  stud3 след промяна:          "; stud3.Output(cout) << endl;
```

Забележете, че `stud3` имаше оценка `2.00` преди присвояването. След него тя е изчезнала — точно както се очаква, защото присвояването заменя цялото съдържание.

8.4 Самоприсвояване и верига

```
// В реален код самоприсвояването идва през псевдоним, а не буквално "a = a".
CStudent& refStud1 = stud1;
stud1 = refStud1;                    // не трябва да разруши обекта

CStudent studA, studB;
studA = studB = stud1;               // работи, защото operator= връща *this
```

Защо през референция, а не направо `stud1 = stud1`; ? Двете са едно и също за програмата, но при буквалния запис компилаторът предупреждава (C4068 при Visual Studio, `-Wself-assign-overloaded` при други). По-важното е, че записът през референция е по-честият случай в реален код — самоприсвояването почти винаги идва скрито, през два указателя или две референции към един обект, и точно затова проверката от стъпка 6 е нужна.

8.5 Растежът на масива

```
CStudent stud4("Мария Иванова", addrVarna, "1111111110", "21621304");
const double arrGrades[] = { 6.00, 5.50, 5.00, 4.50, 4.00, 3.50 };
const size_t nGrades = sizeof(arrGrades) / sizeof(arrGrades[0]);
for (size_t i = 0; i < nGrades; i++)
{
    stud4.AddGrade(arrGrades[i]);
    cout << " след " << (i + 1) << " оценки: брой = " << stud4.GetGradeCount()
        << ", среден успех = " << stud4.GetAverage() << endl;
}
```

Шест оценки при начален капацитет 4 означава едно извикване на `grow` — при петата. Отвън това не личи по нищо и точно така трябва да бъде.

8.6 Проверки за валидност и различни видове променливи

```
stud4.AddGrade(7.00); // извън 2.00–6.00, отхвърля се
cout << " GetGrade(100) = " << stud4.GetGrade(100) << " (невалиден индекс)" << endl;

CStudent studEmpty;
cout << " среден успех на празен студент = " << studEmpty.GetAverage() << endl;

CStudent arrStud[2] = { stud1, stud4 }; // масив, инициализиран с копия
CStudent* pStud = new CStudent(stud1); // динамична памет
delete pStud; // тук се вика деструкторът
const CStudent cStud(stud1); // константен обект
```

Всеки ред тук минава през правилото на тримата. `arrStud[2] = { stud1, stud4 }` вика копиращия конструктор два пъти. `new CStudent(stud1)` — още веднъж, а `delete pStud` — деструктора. Ако някоя от трите функции липсваше или беше сгрешена, точно тези редове щяха да се случат.

Компилиране и стартиране

Клавиши	Действие
<code>Ctrl</code> + <code>Shift</code> + <code>B</code>	Build — само компилира.
<code>F5</code>	Start Debugging. В Debug режим Visual Studio проверява за повреда на динамичната памет — стартирайте задачата от стъпка 2 точно така.
<code>Ctrl</code> + <code>F5</code>	Start Without Debugging — задържа прозореца сам.
<code>F9</code> + <code>F5</code>	Точка на прекъсване в деструктора — вижда се кога точно се вика и колко пъти.

```
cl /EHsc /std:c++17 /utf-8 deepcopy.cpp
deepcopy.exe
```

Как да проверите за течове на памет. Добавете най-отгоре във файла:

```
#define _CRTDBG_MAP_ALLOC
#include <crtdbg.h>
```

и като първи ред в `main`: `_CrtSetDbgFlag(_CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF);`

При стартиране с `F5` в прозореца **Output** се появява списък с неосвободените блокове. Ако всичко е наред, пише `Detected memory leaks!` — *не* се появява. Махнете деструктора и стартирайте пак, за да видите разликата.

Кирилица под Windows

Ако вместо български букви виждате `???`: файлът трябва да е записан като *Unicode (UTF-8 with signature)* — *Codepage 65001* (File → Save As → стрелката до Save → Save with Encoding); компилаторът трябва да получи флаг `/utf-8`; конзолата се превключва с `chcp 65001` или с `system("chcp 65001 > nul");` като първи ред в `main` (нужен е `#include <cstdlib>`).

Пълен код на програмата

Ако някъде сте се загубили, сравнете с този листинг:

```
// Тема 3: Динамична памет в класа. Деструктор, дълбоко копиране,  
// оператор за присвояване. "Правилото на тримата".  
//  
// Надгражда класовете CPerson, CAddress и CStudent от упражнение 2:  
// към CStudent се добавя динамичен масив от оценки.  
  
#include <cctype>      // isdigit  
#include <cstdio>      // getchar  
#include <iostream>  
#include <string>  
using namespace std;  
  
enum Gender  
{  
    MALE,      // мъж  
    FEMALE,    // жена  
    UNKNOWN    // неопределен  
};  
  
// =====  
// CPerson – без промяна спрямо упражнение 2  
// =====  
class CPerson  
{  
protected:  
    string m_strName;  
    Gender m_gender;  
    string m_strEGN;  
  
public:  
    CPerson();  
    CPerson(const string& strName, const string& strGender, const string& strEGN);  
    CPerson(const CPerson& pers);  
  
    void SetName(const string& strName);  
    void SetGender(Gender gender);  
    void SetGender(const string& strGender);  
    bool SetEGN(const string& strEGN);  
  
    const string& GetName() const;  
    Gender GetGender() const;  
    const string& GetEGN() const;  
  
    Gender stringToGender(const string& strGender) const;  
    string genderToString() const;  
    bool isValid(const string& strEGN);  
  
    ostream& Output(ostream& toStream) const;  
};  
  
CPerson::CPerson()  
    : m_strName("Неизвестен"), m_gender(UNKNOWN), m_strEGN("") {}
```

```

CPerson::CPerson(const string& strName, const string& strGender, const string& strEGN)
    : m_strName(strName), m_gender(UNKNOWN), m_strEGN("")
{
    m_gender = stringToGender(strGender);
    SetEGN(strEGN);
}

CPerson::CPerson(const CPerson& pers)
    : m_strName(pers.m_strName), m_gender(pers.m_gender), m_strEGN(pers.m_strEGN) {}

void CPerson::SetName(const string& strName)    { m_strName = strName; }
void CPerson::SetGender(Gender gender)         { m_gender = gender; }
void CPerson::SetGender(const string& strGender) { m_gender = stringToGender(strGender); }

bool CPerson::SetEGN(const string& strEGN)
{
    if (!IsValid(strEGN)) return false;
    m_strEGN = strEGN;
    return true;
}

const string& CPerson::GetName() const { return m_strName; }
Gender CPerson::GetGender() const { return m_gender; }
const string& CPerson::GetEGN() const { return m_strEGN; }

Gender CPerson::stringToGender(const string& strGender) const
{
    if (strGender == "мъж" || strGender == "м" || strGender == "male") return MALE;
    if (strGender == "жена" || strGender == "ж" || strGender == "female") return FEMALE;
    return UNKNOWN;
}

string CPerson::genderToString() const
{
    switch (m_gender)
    {
        case MALE: return "мъж";
        case FEMALE: return "жена";
        default: return "неопределен";
    }
}

bool CPerson::IsValid(const string& strEGN)
{
    if (strEGN.length() != 10) return false;
    for (size_t i = 0; i < strEGN.length(); i++)
        if (!isdigit(static_cast<unsigned char>(strEGN[i]))) return false;

    int nYear = (strEGN[0] - '0') * 10 + (strEGN[1] - '0');
    int nMonth = (strEGN[2] - '0') * 10 + (strEGN[3] - '0');
    int nDay = (strEGN[4] - '0') * 10 + (strEGN[5] - '0');

    if (nMonth >= 1 && nMonth <= 12) nYear += 1900;
    else if (nMonth >= 21 && nMonth <= 32) { nYear += 1800; nMonth -= 20; }
    else if (nMonth >= 41 && nMonth <= 52) { nYear += 2000; nMonth -= 40; }
    else return false;

    int arrDays[12] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };

```

```

bool bLeap = (nYear % 4 == 0 && nYear % 100 != 0) || (nYear % 400 == 0);
if (nMonth == 2 && bLeap) arrDays[1] = 29;
if (nDay < 1 || nDay > arrDays[nMonth - 1]) return false;

const int arrWeights[9] = { 2, 4, 8, 5, 10, 9, 7, 3, 6 };
int nSum = 0;
for (int i = 0; i < 9; i++) nSum += (strEGN[i] - '0') * arrWeights[i];

int nCheck = nSum % 11;
if (nCheck == 10) nCheck = 0;
return nCheck == (strEGN[9] - '0');
}

ostream& CPerson::Output(ostream& toStream) const
{
    toStream << m_strName << " (" << genderToString() << "), ЕГН: "
        << (m_strEGN.empty() ? string("няма") : m_strEGN);
    return toStream;
}

// =====
// CAddress – без промяна спрямо упражнение 2
// =====
class CAddress
{
private:
    string m_strStreet;
    string m_strPostCode;
    string m_strCity;

public:
    CAddress();
    CAddress(const string& strStreet, const string& strPostCode, const string& strCity);
    CAddress(const CAddress& addr);

    const string& GetStreet() const;
    const string& GetPostCode() const;
    const string& GetCity() const;
    void SetStreet(const string& strStreet);
    void SetPostCode(const string& strPostCode);
    void SetCity(const string& strCity);

    ostream& Output(ostream& toStream) const;
};

CAddress::CAddress()
    : m_strStreet("няма улица"), m_strPostCode("0000"), m_strCity("няма град") {}

CAddress::CAddress(const string& strStreet, const string& strPostCode, const string& strCity)
    : m_strStreet(strStreet), m_strPostCode(strPostCode), m_strCity(strCity) {}

CAddress::CAddress(const CAddress& addr)
    : m_strStreet(addr.m_strStreet), m_strPostCode(addr.m_strPostCode),
      m_strCity(addr.m_strCity) {}

const string& CAddress::GetStreet() const { return m_strStreet; }
const string& CAddress::GetPostCode() const { return m_strPostCode; }
const string& CAddress::GetCity() const { return m_strCity; }

```

```

void CAddress::SetStreet(const string& strStreet)    { m_strStreet = strStreet; }
void CAddress::SetPostCode(const string& strPostCode) { m_strPostCode = strPostCode; }
void CAddress::SetCity(const string& strCity)        { m_strCity = strCity; }

ostream& CAddress::Output(ostream& toStream) const
{
    toStream << m_strCity << " " << m_strPostCode << ", " << m_strStreet;
    return toStream;
}

// =====
// ДЕМОНСТРАЦИЯ: клас БЕЗ правилото на тримата
// Показва какво прави генерираният от компилатора копиращ конструктор,
// когато класът държи указател към динамична памет.
// =====
class CBadGrades
{
public:
    double* m_pdGrades;    // указател към динамичен масив
    size_t  m_nCount;

    CBadGrades(size_t nCount)
        : m_pdGrades(new double[nCount]), m_nCount(nCount)
    {
        for (size_t i = 0; i < nCount; i++)
            m_pdGrades[i] = 0.0;
    }

    ~CBadGrades() { delete[] m_pdGrades; }

    // НЯМА копиращ конструктор и НЯМА оператор =.
    // Компиляторът генерира свои – те копират САМИЯ УКАЗАТЕЛ.
};

// =====
// CStudent – надграден: динамичен масив от оценки
// =====
class CStudent : public CPerson
{
private:
    string  m_strFN;
    Address m_address;

    // 1. Динамичната част – това налага правилото на тримата
    double* m_pdGrades;    // указател към масива с оценки
    size_t  m_nCount;      // колко оценки има в момента
    size_t  m_nCapacity;   // за колко има заделено място

    // Помощна: заделя нов масив с двоен размер и пренася старите оценки
    void grow();

public:
    // Конструктори
    CStudent();
    CStudent(const string& strName, const CAddress& addr, const string& strEGN);
    CStudent(const string& strName, const CAddress& addr, const string& strEGN,
              const string& strFN);

    // 2. ПРАВИЛОТО НА ТРИМАТА

```

```

CStudent(const CStudent& stud);           // а) копиращ конструктор
CStudent& operator=(const CStudent& stud); // б) оператор за присвояване
~CStudent();                             // в) деструктор

// Работа с оценките
void AddGrade(double dGrade);
double GetGrade(size_t nIndex) const;
size_t GetGradeCount() const;
double GetAverage() const;

const string& GetFN() const;
const CAddress& GetAddress() const;
void SetFN(const string& strFN);
void SetAddress(const CAddress& addr);

ostream& Output(ostream& toStream) const;
};

// ----- Помощна функция -----
void CStudent::grow()
{
    size_t nNew = (m_nCapacity == 0) ? 4 : m_nCapacity * 2;
    double* pdNew = new double[nNew];

    for (size_t i = 0; i < m_nCount; i++) // пренасяме старите стойности
        pdNew[i] = m_pdGrades[i];

    delete[] m_pdGrades; // освобождаваме стария масив
    m_pdGrades = pdNew;
    m_nCapacity = nNew;
}

// ----- Конструктори -----
CStudent::CStudent()
    : CPerson(), m_strFN("(няма)", m_address(),
      m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0)
{
}

CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN)
    : CPerson(strName, "неопределен", strEGN), m_strFN("(няма)", m_address(addr),
      m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0)
{
}

CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN,
      const string& strFN)
    : CPerson(strName, "неопределен", strEGN), m_strFN(strFN), m_address(addr),
      m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0)
{
}

// ----- а) Копиращ конструктор с ДЪЛБОКО копиране -----
CStudent::CStudent(const CStudent& stud)
    : CPerson(stud), m_strFN(stud.m_strFN), m_address(stud.m_address),
      m_pdGrades(nullptr), m_nCount(stud.m_nCount), m_nCapacity(stud.m_nCapacity)
{
    if (m_nCapacity > 0)
    {

```

```

        m_pdGrades = new double[m_nCapacity];    // СОБСТВЕНА памет
        for (size_t i = 0; i < m_nCount; i++)    // копираме СТОЙНОСТИТЕ
            m_pdGrades[i] = stud.m_pdGrades[i];
    }
}

// ----- б) Оператор за присвояване -----
CStudent& CStudent::operator=(const CStudent& stud)
{
    // 1. Защита от самоприсвояване: stud1 = stud1;
    if (this == &stud)
        return *this;

    // 2. Базовата и вградената част – чрез техните оператори =
    CPerson::operator=(stud);
    m_strFN = stud.m_strFN;
    m_address = stud.m_address;

    // 3. Старата памет се освобождава ПРЕДИ да се заеме нова
    delete[] m_pdGrades;
    m_pdGrades = nullptr;

    // 4. Дълбоко копиране, както в копиращия конструктор
    m_nCount = stud.m_nCount;
    m_nCapacity = stud.m_nCapacity;
    if (m_nCapacity > 0)
    {
        m_pdGrades = new double[m_nCapacity];
        for (size_t i = 0; i < m_nCount; i++)
            m_pdGrades[i] = stud.m_pdGrades[i];
    }

    // 5. Връщаме *this, за да работи веригата a = b = c;
    return *this;
}

// ----- в) Деструктор -----
CStudent::~~CStudent()
{
    delete[] m_pdGrades;    // delete[], защото е заделено с new[]
    m_pdGrades = nullptr;
}

// ----- Работа с оценките -----
void CStudent::AddGrade(double dGrade)
{
    if (dGrade < 2.0 || dGrade > 6.0)    // проверка за валидност
    {
        cout << " (отхвърлена оценка: " << dGrade << ")" << endl;
        return;
    }
    if (m_nCount == m_nCapacity)
        grow();
    m_pdGrades[m_nCount] = dGrade;
    m_nCount++;
}

double CStudent::GetGrade(size_t nIndex) const
{

```

```

    if (nIndex >= m_nCount)
        return 0.0; // невалиден индекс
    return m_pdGrades[nIndex];
}

size_t CStudent::GetGradeCount() const { return m_nCount; }

double CStudent::GetAverage() const
{
    if (m_nCount == 0)
        return 0.0; // защита от деление на нула
    double dSum = 0.0;
    for (size_t i = 0; i < m_nCount; i++)
        dSum += m_pdGrades[i];
    return dSum / m_nCount;
}

// ----- Четене / презапис -----
const string& CStudent::GetFN() const { return m_strFN; }
const CAddress& CStudent::GetAddress() const { return m_address; }

void CStudent::SetFN(const string& strFN) { m_strFN = strFN; }
void CStudent::SetAddress(const CAddress& addr) { m_address = addr; }

// ----- Извеждане -----
ostream& CStudent::Output(ostream& toStream) const
{
    CPerson::Output(toStream);
    toStream << ", ф.н.: " << m_strFN << ", адрес: ";
    m_address.Output(toStream);

    toStream << ", оценки: ";
    if (m_nCount == 0)
        toStream << "(няма)";
    else
    {
        for (size_t i = 0; i < m_nCount; i++)
            toStream << m_pdGrades[i] << (i + 1 < m_nCount ? " " : "");
        toStream << " | среден успех: " << GetAverage();
    }
    return toStream;
}

// =====
// Главна функция
// =====
int main()
{
    cout.setf(ios::fixed);
    cout.precision(2);

    CAddress addrVarna("Studentska #1", "9010", "Varna");

    // ===== А. Проблемът: плитко копиране =====
    cout << "===== А. Плитко копиране (клас без правилото на тримата) =====" << endl;
    {
        CBadGrades a(3);
        a.m_pdGrades[0] = 5.50;
    }
}

```

```

        CBadGrades b(a);           // генерираният копиращ конструктор

        cout << "  a[0] преди промяната на b: " << a.m_pdGrades[0] << endl;
        b.m_pdGrades[0] = 2.00;    // пипаме CAMO b...
        cout << "  a[0] след промяната на b: " << a.m_pdGrades[0]
            << "  <-- промени се и a!" << endl;

        cout << "  адрес на a.m_pdGrades: " << (a.m_pdGrades == b.m_pdGrades
            ? "СЪЩИЯТ като на b" : "различен от b") << endl;

        // Обезвреждаме двойното изтриване, за да не се срие програмата:
        b.m_pdGrades = nullptr;
    }

    // ===== Б. Решението: дълбоко копиране =====
    cout << endl << "===== Б. Дълбоко копиране (CStudent) =====" << endl;

    CStudent stud1("Иван Петров", addrVarna, "1111111110", "21621301");
    stud1.AddGrade(5.50);
    stud1.AddGrade(4.00);
    stud1.AddGrade(6.00);
    cout << "  stud1: "; stud1.Output(cout) << endl;

    cout << "---- Копиращ конструктор ----" << endl;
    CStudent stud2(stud1);           // копиращ конструктор
    stud2.SetName("Елена Димитрова");
    stud2.SetFN("21621302");
    stud2.AddGrade(3.00);           // добавяме CAMO в копието
    cout << "  stud1 (оригинал): "; stud1.Output(cout) << endl;
    cout << "  stud2 (копие):      "; stud2.Output(cout) << endl;

    cout << "---- Оператор за присвояване ----" << endl;
    CStudent stud3("Георги Стоянов", addrVarna, "1111111104", "21621303");
    stud3.AddGrade(2.00);
    cout << "  stud3 преди: "; stud3.Output(cout) << endl;
    stud3 = stud1;                  // оператор =
    cout << "  stud3 след stud3 = stud1: "; stud3.Output(cout) << endl;

    stud3.AddGrade(4.50);           // промяна само в stud3
    cout << "  stud1 след промяна на stud3: "; stud1.Output(cout) << endl;
    cout << "  stud3 след промяна:      "; stud3.Output(cout) << endl;

    cout << "---- Самоприсвояване ----" << endl;
    // В реален код самоприсвояването идва през псевдоним, а не буквално "a = a".
    CStudent& refStud1 = stud1;
    stud1 = refStud1;              // не трябва да разруши обекта
    cout << "  stud1 след stud1 = refStud1: "; stud1.Output(cout) << endl;

    cout << "---- Верига от присвоявания ----" << endl;
    CStudent studA, studB;
    studA = studB = stud1;         // работи, защото operator= връща *this
    cout << "  studA: "; studA.Output(cout) << endl;

    // ===== В. Растежът на масива =====
    cout << endl << "===== В. Растеж на динамичния масив =====" << endl;
    CStudent stud4("Мария Иванова", addrVarna, "1111111110", "21621304");
    const double arrGrades[] = { 6.00, 5.50, 5.00, 4.50, 4.00, 3.50 };
    const size_t nGrades = sizeof(arrGrades) / sizeof(arrGrades[0]);
    for (size_t i = 0; i < nGrades; i++)

```

```

{
    stud4.AddGrade(arrGrades[i]);
    cout << " след " << (i + 1) << " оценки: брой = " << stud4.GetGradeCount()
        << ", среден успех = " << stud4.GetAverage() << endl;
}

// ===== Г. Проверки за валидност =====
cout << endl << "===== Г. Проверки =====" << endl;
cout << " AddGrade(7.00):" << endl;
stud4.AddGrade(7.00); // извън 2.00-6.00
cout << " AddGrade(1.00):" << endl;
stud4.AddGrade(1.00); // извън 2.00-6.00
cout << " брой оценки остава: " << stud4.GetGradeCount() << endl;
cout << " GetGrade(0) = " << stud4.GetGrade(0) << endl;
cout << " GetGrade(100) = " << stud4.GetGrade(100) << " (невалиден индекс)" << endl;

CStudent studEmpty;
cout << " среден успех на празен студент = " << studEmpty.GetAverage() << endl;

// ===== Д. Обекти от различни видове =====
cout << endl << "===== Д. Различни видове променливи =====" << endl;

CStudent arrStud[2] = { stud1, stud4 }; // масив, инициализиран с копия
for (int i = 0; i < 2; i++)
{
    cout << " [" << i << "] ";
    arrStud[i].Output(cout) << endl;
}

CStudent* pStud = new CStudent(stud1); // динамична памет
cout << " през указател: ";
pStud->Output(cout) << endl;
delete pStud; // тук се вика деструкторът
pStud = nullptr;

const CStudent cStud(stud1); // константен обект
cout << " константен: ";
cStud.Output(cout) << endl;

// Задържа конзолния прозорец отворен при стартиране с F5 (debug)
getchar();
return 0;
}

```

Очакван изход

===== А. Плитко копиране (клас без правилото на тримата) =====

a[0] преди промяната на b: 5.50
a[0] след промяната на b: 2.00 <-- промени се и a!
адрес на a.m_pdGrades: СЪЩИЯТ като на b

===== Б. Дълбоко копиране (CStudent) =====

stud1: Иван Петров (неопределен), ЕГН: 1111111110, ф.н.: 21621301, адрес: Varna 9010, Studentska #1, оценки: 5.50 4.00 6.00 | среден успех: 5.17

--- Копиращ конструктор ---

stud1 (оригинал): Иван Петров (неопределен), ЕГН: 1111111110, ф.н.: 21621301, адрес: Varna

```

9010, Studentska #1, оценки: 5.50 4.00 6.00 | среден успех: 5.17
stud2 (копие): Елена Димитрова (неопределен), ЕГН: 111111110, ф.н.: 21621302, адрес:
Varna 9010, Studentska #1, оценки: 5.50 4.00 6.00 3.00 | среден успех: 4.62
--- Оператор за присвояване ---
stud3 преди: Георги Стоянов (неопределен), ЕГН: 1111111104, ф.н.: 21621303, адрес: Varna
9010, Studentska #1, оценки: 2.00 | среден успех: 2.00
stud3 след stud3 = stud1: Иван Петров (неопределен), ЕГН: 111111110, ф.н.: 21621301, адрес:
Varna 9010, Studentska #1, оценки: 5.50 4.00 6.00 | среден успех: 5.17
stud1 след промяна на stud3: Иван Петров (неопределен), ЕГН: 111111110, ф.н.: 21621301,
адрес: Varna 9010, Studentska #1, оценки: 5.50 4.00 6.00 | среден успех: 5.17
stud3 след промяна: Иван Петров (неопределен), ЕГН: 111111110, ф.н.: 21621301,
адрес: Varna 9010, Studentska #1, оценки: 5.50 4.00 6.00 4.50 | среден успех: 5.00
--- Самоприсвояване ---
stud1 след stud1 = refStud1: Иван Петров (неопределен), ЕГН: 111111110, ф.н.: 21621301,
адрес: Varna 9010, Studentska #1, оценки: 5.50 4.00 6.00 | среден успех: 5.17
--- Верига от присвоявания ---
studA: Иван Петров (неопределен), ЕГН: 111111110, ф.н.: 21621301, адрес: Varna 9010,
Studentska #1, оценки: 5.50 4.00 6.00 | среден успех: 5.17

===== В. Растеж на динамичния масив =====
след 1 оценки: брой = 1, среден успех = 6.00
след 2 оценки: брой = 2, среден успех = 5.75
след 3 оценки: брой = 3, среден успех = 5.50
след 4 оценки: брой = 4, среден успех = 5.25
след 5 оценки: брой = 5, среден успех = 5.00
след 6 оценки: брой = 6, среден успех = 4.75

===== Г. Проверки =====
AddGrade(7.00):
(отхвърлена оценка: 7.00)
AddGrade(1.00):
(отхвърлена оценка: 1.00)
брой оценки остава: 6
GetGrade(0) = 6.00
GetGrade(100) = 0.00 (невалиден индекс)
среден успех на празен студент = 0.00

===== Д. Различни видове променливи =====
[0] Иван Петров (неопределен), ЕГН: 111111110, ф.н.: 21621301, адрес: Varna 9010, Studentska
#1, оценки: 5.50 4.00 6.00 | среден успех: 5.17
[1] Мария Иванова (неопределен), ЕГН: 111111110, ф.н.: 21621304, адрес: Varna 9010,
Studentska #1, оценки: 6.00 5.50 5.00 4.50 4.00 3.50 | среден успех: 4.75
през указател: Иван Петров (неопределен), ЕГН: 111111110, ф.н.: 21621301, адрес: Varna 9010,
Studentska #1, оценки: 5.50 4.00 6.00 | среден успех: 5.17
константен: Иван Петров (неопределен), ЕГН: 111111110, ф.н.: 21621301, адрес: Varna 9010,
Studentska #1, оценки: 5.50 4.00 6.00 | среден успех: 5.17

```

Чести грешки

Симптом	Причина и решение
HEAP CORRUPTION DETECTED при излизане от блок	Двойно <code>delete[]</code> върху един адрес — липсва дълбоко копиране. Проверете дали копиращият конструктор заделя нова памет.
Срив в деструктора още при първия обект	<code>m_pdGrades</code> не е инициализиран с <code>nullptr</code> в някой конструктор (стъпка 3).

Промяна в копието се вижда в оригинала	Използва се генерираният копиращ конструктор — вашият не се вика. Проверете сигнатурата: трябва да е точно <code>const CStudent&</code> .
Обектът се изпразва след <code>a = a;</code>	Липсва проверката <code>if (this == &stud) return *this;</code> (стъпка 6).
<code>Detected memory leaks!</code> в Output	Липсва деструктор, или <code>operator=</code> не трие старата памет преди да заеме нова.
<code>C2582 : 'operator =' function is unavailable</code>	Класът има <code>const</code> или референтна член-променлива — те не могат да се присвояват.
<code>C4267 : conversion from 'size_t' to 'int'</code>	Някъде се смесват <code>int</code> и <code>size_t</code> . Използвайте <code>size_t</code> за броячи на елементи.
<code>a = b = c;</code> не се компилира	<code>operator=</code> връща <code>void</code> вместо <code>CStudent&</code> .
Безкрайна рекурсия / <i>Stack overflow</i>	В <code>CStudent::Output</code> е написано <code>Output(...)</code> вместо <code>CPerson::Output(...)</code> — грешката от упражнение 2.

Правилото на тримата — обобщение

Ако класът има нужда от **едно** от трите — деструктор, копиращ конструктор, оператор за присвояване — почти сигурно има нужда и от **трите**. Признакът е винаги един и същ: класът притежава ресурс, който не се управлява сам. Най-често указател към динамична памет, но също отворен файл, мрежова връзка или заключване.

Обратното също е вярно и е по-полезно: **ако класът няма нито един такъв ресурс, не пишете нито една от трите**. Генерираните от компилатора са коректни, по-кратки и не могат да се сгрешат. Затова `CPerson` и `CAddress` минават без деструктор — техните `string` членове се грижат сами за себе си.

Какво следва

Упражнение 4	Приятелски функции и оператори: <code>Output</code> се заменя с <code>operator<<</code> , за да се пише <code>cout << stud;</code> ; оператори за сравнение по възраст; извеждане във файл.
Упражнение 5	Виртуални функции — решението на проблема с указателя към базов клас; множествено наследяване; четене от файл.
Упражнение 9	<code>std::vector</code> — контейнер, който прави всичко от това упражнение вместо вас и не може да се сгреша.